Architecture client–serveur pour le filtrage d'images en C

Rapport de Projet

Élèves :
Mateo Damiani
Yöen Pesquer

Enseignants :
Mr Djelloul Ziadi
Mr Stephane Herauville

30 décembre 2025

Table des matières

1 **Introduction** 2
- a Utilisation . . . 2

2 **Commun** 2
- a Structures utilisées . . . 2
  - I filter_request . . . 2
  - II shared_data . . . 3
- b Code Commun . . . 3

3 **Client** 4

4 **Serveur** 7
- a Serveur . . . 7
- b Worker . . . 8
- c BMP . . . 10
  - I Structures . . . 10
  - II Fonctions . . . 11

5 **Ajouts optionels** 12
- a Filtres . . . 12
- b Démon . . . 12
- c Sécurité . . . 13
  - I Déni de service . . . 13
  - II Limitation de worker . . . 13
- d Signaux . . . 13
  - I Sigchild . . . 13
  - II Sigusr1 . . . 13
  - III Sigterm . . . 14

6 **Améliorations possibles** 14

7 **Conclusion** 14

1 Introduction

L'objectif de ce projet est de créer une architecture client / serveur permettant aux clients de soumettre des requêtes de filtrage d'images au serveur. Pour cela, le projet a nécessité plusieurs programmes C afin de satisfaire les contraintes exigées. Le projet est donc divisé en 3 parties, le côté client, le côté serveur et enfin le code commun aux deux parties.

a Utilisation

Il faut premièrement lancer le serveur, pour cela il suffit de d'exécuter dans un terminal l'exécutable du serveur, le terminal n'a pas besoin de rester ouvert.
Le serveur est désormais actif ! En cas de changement au fichier de configuration, il faut réaliser la commande "kill -sigusr1 <pid du serveur>" (on peut obtenir le pid grâce à : ps -ef | grep serveur) pour mettre le serveur à jour. Maintenant que le serveur est lancé, on peut réaliser la commande "./client <image.bmp> <id filtre>" avec <image.bmp> le chemin de l'image à filtrer et <id filtre> l'identifiant du filtre utilisé. En réalisant la commande "./client -?", une aide sera affichée expliquant le programme client et montrant quel id correspond à quel filtre.

2 Commun

a Structures utilisées

I filter_request

Filter_request est la structure contenant toutes les informations du client que le serveur utilisera par la suite pour traiter l'image.

Structure filter_request

```c
typedef struct {
    pid_t pid;              //identifient du client
    char chemin [256];      //chemin absolu vers l'image
    int filtre;             //numéro du filtre choisi
    int parametres [5];     //paramètres du filtre
    unsigned int timeout; // attente max du worker (serveur)
} filter_request;
```

II shared_data

Shared_data est la SHM partagée entre le serveur et tous les client.

Structure shared_data

```
typedef struct {
    size_t read;                //le curseur de lecture du serveur
    size_t write;               //le curseur d'écriture des clients
    filter_request t[SHM_SIZE]; //un tableau de requêtes clients
} shared_data;
```

b Code Commun

Commun.c rassemble toutes les fonctions utilisé par le serveur et les clients. Il contient, les fonctions :

V : réalise sem_post.

P : réalise sem_wait, return si stopé par un signal.

get_shared_memory : crée ou ouvre une shm en fonction que l'on est le serveur ou un client.

is_sem_incorrect : vérifie la réussite de l'action de get_semaphore, libère les ressources en cas d'erreur.

get_semaphore : crée ou ouvre un sémaphore en fonction que l'on est le serveur ou un client.

3 Client

Le code client est relativement direct. Premièrement, toutes les variables qui seront utilisées sont initialisées à nullptr ou -1 afin de simplifier les libérations / fermeture en fin de programme. Ensuite, une série de test est réalisée afin de savoir si la requête du client est valide et récupérer certaines informations sur la requête comme le filtre demandé. La variable fifo_created permet de supprimer la FIFO en cas d'erreur si la FIFO a déjà été crée / ouverte.

Série de tests

```
// verify number of arguments
if (argc < 3) {
  goto error_arguments;
}

//get image full path
if (realpath(argv[1], full_path) == nullptr) {
  perror("realpath");
  goto error;
}

if (stat(full_path, &st) == -1) {
  perror("stat");
  goto error;
}

// Vérification de la taille du fichier
off_t taille = st.st_size;
if (st.st_size <= 0 || taille > 100L * 1024 * 1024) {
    fprintf(stderr, "Taille de fichier invalide: \%ld\n", st.st_size);
    goto error;
}

// get filter
int filter_id = atoi(argv[2]);
if (filter_id <= 0 || filter_id > MAX_FILTER) {
  goto error_filter;
}

// get extension
char *extension = full_path + strlen(full_path) - 3;
if (strcmp(extension, "bmp") != 0) {
  goto error_extension;
}
```

Si tous les tests sont valides, alors les sémaphores et les segments de mémoires sont ouverts. La structure de requête est crée et initialisée selon les valeurs données par l'utilisateur. Et on initialise un signal, son objectif sera de notifier le client en cas de temps d'attente du serveur trop long (Time Out) et permettre au code client de se stopper dans des bonnes conditions.

Et enfin vient le moment d'envoyer la requête au serveur, en utilisant les sémaphores, on peut s'assurer que le segment de mémoire partagée est disponible et donc envoyer la requête.

Envoie de la requête

```
// send request if there is space in shm
P(full);
P(mutex);
  data->t[data->write] = fr;
  data->write = (data->write + 1) % *shm_info;
V(mutex);
V(empty);
```

Ensuite, on ouvre la FIFO afin de pouvoir obtenir un code de succés (le serveur a réussi à filtrer l'image). C'est à ce moment là que le signal prend de l'intérêt car il permet d'établir un temps de réponse, empêchant ainsi le blocage du client si le serveur n'est pas connecté à la FIFO.

La requête a été envoyé et un code de succés a été envoyé, le client peut maintenant lire l'image. Une image possède une taille connu du programme mais on ne sait pas combien d'octets sera lu par le client en un appel à read. On utilise donc la valeur de retour de la fonction read afin de connaître le nombre d'octets lu. On stocke le nombre total d'octets lus par l'utilisateur pour s'assurer qu'une fois sortie de la boucle de lecture, l'intégralité de l'image a été lu. Après la lecture, la FIFO est fermée et désynchronisée car elle n'est plus utilisée dans le reste du programme client.

Boucle de lecture de l'image

```
size_t total_read = 0;
alarm(TIMEOUT_DATA);
while (total_read < (size_t)st.st_size) {
  size_t remaining = (size_t)st.st_size - total_read;
  if (img_data == nullptr) {
    goto error;
  }
  ssize_t bytes = read(fifo_fd, img_data + total_read, remaining);
  if (bytes == -1) {
    if (errno == ETIMEDOUT) {
      fprintf(stderr, "Timeout: données incomplète recu\n");
    } else {
      perror("read");
```

```
    }
    free(img_data);
    img_data = nullptr;
    goto error;
  }
  if (bytes == 0) {
    fprintf(stderr, "EOF prématuré\n");
    free(img_data);
    img_data = nullptr;
    goto error;
  }
  total_read += (size_t)bytes;
}
close(fifo_fd);
fifo_fd = -1;
unlink(fifo_name);
```

Si l'image a été lu entièrement, elle est alors réécrite dans un nouveau fichier crée pour stocker l'image filtrée.

Ecriture de l'image

```
char new_path[PATH_MAX + 9];
snprintf(new_path, sizeof(new_path), "\%s_filtered", full_path);
fd_out = open(new_path, O_WRONLY | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);
if (fd_out == -1) {
    perror("open output");
    free(img_data);
    img_data = nullptr;
    goto error;
}

if (write(fd_out, img_data, (size_t)st.st_size) != st.st_size) {
    fprintf(stderr, "Erreur écriture fichier\n");
    free(img_data);
    img_data = nullptr;
    goto error;
}
```

Pour finir, le client va fermer / libérer les structures et variables qu'il a utilisé dans son programme. C'est ainsi que le client envoie une requête au serveur et reçoit l'image filtrée.

4 Serveur

a Serveur

Le serveur a un fonctionnement différent du client, il possède sa propre structure de configuration server_config_t et il est programmé de manière à être actif en continu. De plus, le serveur ne filtre pas directement l'image, il crée un processus fils pour appeler un worker qui est spécialisé dans le filtre de l'image.

Structure de configurations du serveur

```
typedef struct {
  unsigned int max_threads_used; // Nombre maximum de threads
  unsigned int max_request;      // Nombre maximum de requêtes
  unsigned int max_workers;      // Nombre maximum de worker
  unsigned int waiting_time;     // Temps d'attente
} server_config_t;
```

Au lancement du serveur, le fichier de configuration est ouvert afin d'initialiser la structure server_config_t puis les sémaphores et segments de mémoire partagés sont ouverts. A ce moment-là, le serveur est opérationnel et il rentre dans une boucle continu d'attente. Le serveur va attendre qu'une requête soit envoyé. Durant la boucle, des tests auront lieu pour vérifier si il faut recharger le fichier de configuration ou si la commande d'arrêt du serveur a été lancée grâce au "handler" des signaux usr1 et term. Les appels aux fonctions P et V des sémaphores permettent un nombre limité de requêtes (empêche la saturation du serveur) mais aussi un accés unique au segment de mémoire partagée.

Gestion de la réception d'une requête

```
P(sem_worker);
  // tests de signals
  if (reload == 1) {
    if (reload_config(&config) == -1) {
      goto dispose;
    }
    goto boucle;
  }
  if (do_stop == 1) {
    goto dispose;
  }
  P(empty);
  // tests de signals
P(mutex);
  // tests de signals
```

```
    filter_request fr;
    size_t read = data->read;
    fr.pid = data->t[read].pid;
    snprintf(fr.chemin, sizeof(fr.chemin), "%s", data->t[read].chemin);
    fr.filtre = data->t[read].filtre;
    fr.timeout = config.waiting_time;
    data->read = (read + 1) % *shm_info;
  V(mutex);
  V(full);
```

Ici, P(sem_worker) permet de s'assurer que le nombre de worker actifs n'est pas supérieur à celle spécifier dans la structure de configuration. Ensuite P(empty) sert à s'assurer que le nombre de requêtes n'est pas supérieur au maximum. Et enfin, P(Mutex) permet de prendre restreindre l'accés au segment de mémoire partagé. Toutes les conditions sont réunis pour l'écriture des données essentielles au worker dans la shm. Une fois écrite, le serveur crée un processus fils appellant la fonction de worker puis V(sem_worker) "libère" un emplacement de worker pour les futures requêtes.

Une fois le signal sigterm reçu, un ensemble de fonction est appelé afin de libérer / fermer les sémaphores et structures utilisées.

b Worker

Le worker est crée par le serveur lorsqu'il reçoit une requête client. Son rôle est d'appliquer un filtre sur une image de type BMP, pour ce faire il appelle une fonction principale nommée "worker".

La fonction crée une shm de l'image, elle copie l'image originale dans la zone partagée, elle continue en allouant les structure BMPData et un tableau de thread_workspace puis fait appel aux fonctions d'initialisation contenue dans bmp.c. Enfin la fonction crée les threads et leurs assigne comme fonction "threadFunc" et comme paramètre le thread_workspace du même indice que le thread.

Gestion des threads

```
//Manage Threads
threadArray = calloc(THREAD_NB, sizeof(*threadArray));
if (threadArray == nullptr) {
    goto send_error;
}
for(int k = 0; k < THREAD_NB; k++) {
    pthread_t tid;
    if (pthread_create(&tid, nullptr, threadFunc, &threadArgs[k]) != 0) {
        perror("pthread_create");
```

```
        //Attendre threads déjà créés
        for(int j = 0; j < k; j++) {
            pthread_join(threadArray[j], nullptr);
        }
        goto send_error;
    }
    threadArray[k] = tid;
}
for (int k = 0; k < THREAD_NB; k++) {
    pthread_join(threadArray[k], nullptr);
}
```

ThreadFunc : En fonction du th->filtre, un appel à un filtre différent est effectué.

ThreadFunc

```
void *threadFunc(void *arg) {
  thread_workspace *th = (thread_workspace*) arg;
  switch(th->filtre) {
    case 1:
      apply_grayscale(th);
      break;
    case 2:
      apply_redscale(th);
      break;
    case 3:
      apply_greenscale(th);
      break;
    case 4:
      apply_bluescale(th);
      break;
    case 5:
      apply_pinkscale(th);
      break;
    case 6:
      apply_transparency(th);
      break;
    case 7:
      apply_negative(th);
      break;
    case 8:
      th->filtre = (rand() \% 7) + 1;
      threadFunc(th);
      break;
```

```
  }
  return nullptr;
}
```

A la fin du travail de tous les threads, le worker essaye de se connecter à la FIFO de réponse du client. Tout comme le client, le worker écoute le signal sigalarm, ce qui lui permet de se terminer proprement si le client ne s'est pas connecté à la FIFO.
Si la connexion s'est bien passé, le worker envoie un message d'échec ou succés au client selon si il s'est produit une erreur lors de l'application du filtre ou non. Si c'est un succès, le worker envoie l'image modifié au segment de mémoire partagée. Après cela fait, il libère les ressources dont il a eu besoin pour travailler et se termine.

c BMP

Le fichier bmp.c rassemble toutes les fonctions et les structures qui permettent le parcours dans une image de type bmp et d'y appliquer différents filtres.

I Structures

BITMAPFILEHEADER est la structure qui représente les premières informations des images de type BMP, qui servent à la manipulation des pixels de l'image.

Bit Map File Header

```
struct BITMAPFILEHEADER {
  uint16_t bfType;
  uint32_t bfSize;
  uint16_t bfReserved1;
  uint16_t bfReserved2;
  uint32_t bfOffBits;
  uint32_t biSize;
  int32_t  biWidth;
  int32_t  biHeight;
  uint16_t biPlanes;
  uint16_t biBitCount;
  uint32_t biCompression;
};
```

BMPData est la structure qui mémorise des résultats pré-calculés de valeurs qui servent au parcours de pixel d'une image donnée. Cela permet d'éviter de re-calculer à chaque déplacement de pixel les informations du pixel comme le padding.

BMP Data

```
struct BMPData{
  uint8_t *shm;           // mémoire partagée (début du fichier BMP)
  uint8_t *pixelArray;    // pointeur vers les pixels dans shm

  int32_t width;
  int32_t height;
  uint32_t bytesPerPixel;
  uint32_t rowSize;
  uint32_t padding;
  uint32_t dataOffset;    // bfOffBits
};
```

Thread_workspace est la structure donnée à 1 thread, elle a pour rôle de définir la zone d'action du thread sur l'image, de contenir le filtre à appliqué et la structure BMPData de valeur pré-calculé

Thread Workspace

```
typedef struct {
  BMPData *BMPData;
  uint32_t startOffset;
  uint32_t pixelCount;
  int filtre;
} thread_workspace;
```

II Fonctions

initBMPData : Alloue une structure BMPData et initialise tous les champs.

divideImageForThreads : Initialise les structures thread_workspace de tw, elles contiendrons l'indice de début et de fin du tableau de pixel stocké dans BMPData.

pixelIndexToOffset : Calcule un indice à partir du tableau de pixel à partir d'un offset.

move_once : Avance d'un pixel à partir de l'offset donné.

apply_grayscale : Fonction appelé par un thread, modifie les valeurs RGB du pixel pour appliqué le filtre gris, puis avance au pixel suivant jusqu'à atteindre l'indice pixelCount. (même chose pour les autres filtres définits dans ce projet)

5 Ajouts optionels

a Filtres

Durant l'exécution du worker, les threads feront appels à la fonction de filtre afin de filtrer leurs propres portions de l'image. Une image en BMP peut se représenter comme un tableau de pixel (pixel étant comme un tableau de 3 uint8_t pour les composantes RGB). Ainsi, le thread a besoin d'un offset (pixel de début) et un nombre de pixel n à filtrer. La fonction de filtre va donc parcourir n pixels à partir de cet offset. Et pour chaque pixel individuel, elle va appliquer une formule sur les composantes RGB afin d'appliquer le filtre souhaité.

Liste des filtres implémentés :

Filtre Gris (grayscale)
Filtre Rouge (redscale)
Filtre Vert (greenscale)
Filtre Blue (bluescale)
Filtre Transparent (transparency)
Filtre Négatif (negate)

Enfin, nous avons implémenter un filtre "aléatoire" qui va assigner à chaque thread un filtre aléatoire parmi ceux listés.

b Démon

Afin de devenir un démon, un processus doit se détacher complètement de son environnement d'exécution initial et fonctionner en arrière-plan de manière autonome. Cela commence par un premier fork(), qui permet au processus parent de se terminer afin que le processus fils ne soit plus lié au shell. Le fils crée ensuite une nouvelle session avec setsid(), ce qui le détache du terminal de contrôle et le rend leader de session.

Un second fork() est alors effectué pour garantir que le processus ne pourra jamais redevenir leader de session et ainsi éviter toute ré-association à un terminal. Le démon réinitialise ensuite le masque de création de fichiers (umask), puis change son répertoire de travail vers la racine (/) afin de ne pas bloquer un système de fichiers monté.

Tous les descripteurs de fichiers ouverts sont fermés, et les flux standards (stdin, stdout, stderr) sont redirigés vers /dev/null, assurant que le démon ne produise aucune sortie involontaire. Le processus installe ensuite des gestionnaires de signaux pour permettre un arrêt propre (SIGTERM), une interaction contrôlée (SIGUSR1) et éviter la création de zombies (SIGCHLD avec SA_NOCLDWAIT).

Enfin, le démon bloque tous les signaux non nécessaires, ne conservant que ceux explicitement gérés.

c Sécurité

I Déni de service

Le déni de service consite à bloquer un processus indéfiniment.
Exemple : Le serveur possède 10 worker au maximum, 10 client envoient une requête mais ne se connectent jamais à leurs FIFO de réponse.
Résultat, le serveur se retrouve dans un état bloqué tant que les clients ne lisent pas leurs FIFO

Afin d'empêcher cela, le serveur et les clients utilisent le signal "alarm" afin de se terminer si aucune réponse n'a été donnée par l'autre après un certain délai.

II Limitation de worker

Par sécurité, le serveur possède une limite de worker maximale définie par SHM_SIZE (/Commun/types.h). Il est possible de limiter le nombre de worker de 1 à max_workers dans le fichier de configuration du serveur.

Pourquoi limiter le nombre de requêtes ?
Si il n'y a pas de limite, dans le cas où l'ordonanceur ne donne jamais la main aux worker, il serait possible de saturer la mémoire du serveur par des worker, ce qui ralenti fortement le processus, voir peux causer des échecs d'allocation mémoire.

d Signaux

I Sigchild

Le serveur définie le flag signal sigchld en SA_NOCLDWAIT et le handler en SIG_IGN. Cela signifie que dorénavant, il ignore le signal sigchld et que c'est au noyau de gérér les processus fils quand ils sont terminés et d'empêcher les zombies.

sigchld

```
struct sigaction sa;
sa.sa_handler = SIG_IGN;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_NOCLDWAIT;
sigaction(SIGCHLD, &sa, nullptr);
```

II Sigusr1

A la réception du signal sigusr1, le serveur est dirigé vers la fonction sigusr1_handler qui a pour rôle de passer à 1 la variable globale "reload" de manière atomique et d'effectuer un sem_post(full) qui aura pour conséquence de débloquer le serveur s'il l'était.

sigusr1_handler

```
void sigusr1_handler(int sig) {
  (void)sig;
  reload = 1;
  sem_post(empty);
  sem_post(sem_worker);
}
```

III Sigterm

A la réception du signal sigterm, le serveur est dirigé vers la fonction sigterm_handler qui a pour rôle de passer à 1 la variable globale "do_stop" de manière atomique et d'effectuer un sem_post(full) qui aura pour conséquence de débloquer le serveur s'il l'était.
A la suite du handler, le serveur se dirigera vers l'étiquette dispose qui va libérer toutes les ressources utilisé par le serveur (sémaphores, shm), et se termine.

sigterm_handler

```
void sigterm_handler(int sig) {
  (void)sig;
  do_stop = 1;
  sem_post(empty);
  sem_post(sem_worker);
}
```

6 Améliorations possibles

Une amélioration possible serait d'étendre le projet pour traiter d'autres formats d'images, notamment PNG et JPG. Cela permettrait au programme de devenir plus polyvalent et compatible avec un plus large éventail de fichiers.

7 Conclusion

En conclusion, ce projet nous a permit de mettre en exécution toutes les connaissances que nous avons travaillé pendant le semestre. Notamment, la gestion des sémaphores, des segments de mémoire partagés, des FIFO et des signaux. A travers ce projet, on a crée une architecture répondant aux contraintes demandés. Mais nous sommes allés plus loin avec des filtres supplémentaires, l'implémentation démon avec fichier de configuration et les mesures de sécurité prises.